Chapter: Optimization

Lecture Notes on Unsupervised Learning

Based on lectures by Peter Orbanz

Contents

1 Motivation

The way machine learning applies models to data can roughly be categorised into two approaches:

Optimization. The solution is a specific value. We define mathematically what a "good" solution is and search for the best one.

Simulation. The solution is given by a distribution (e.g. the distribution itself, an expectation, etc.). We approximate the solution by samples drawn from the distribution.

Remark 1.1 (Extremal Principles). Many core methods in machine learning are instances of optimization:

Purpose	**Objective function**
Maximum likelihood estimation	Likelihood
Empirical risk minimization	Error rate (empirical risk)
Variational inference	Distance between approximate and true posterior

This list is far from exhaustive.

2 Terminology

Definition 2.1 (Min and Argmin)**.** For a function $f\colon \mathbb{R}^d \to \mathbb{R}$,

$$\min_x f(x) = \text{smallest value of } f(x) \text{ over all } x, \tag{1}$$

$$\arg\min_x f(x) = \text{value of } x \text{ for which } f(x) \text{ is minimal.} \tag{2}$$

If f depends on several arguments, we may minimize over a subset:

$$\min_x f(x, y) = \text{smallest value of } f(x, y) \text{ over all } x, \text{ with } y \text{ fixed.}$$

Definition 2.2 (Optimization Problem)**.** For a given function $f\colon \mathbb{R}^d \to \mathbb{R}$, a problem of the form

$$\text{find} \quad x^* := \arg\min_x f(x)$$

is called a **minimization problem**. If $\arg\min$ is replaced by $\arg\max$, it is a **maximization problem**. Minimization and maximization problems are collectively referred to as **optimization problems**.

Remark 2.3 (Minimization vs. Maximization). For any function f,

$$\min_x f(x) = -\max_x\big(-f(x)\big) \qquad \text{and} \qquad \arg\min_x f(x) = \arg\max_x\big(-f(x)\big). \tag{3}$$

Hence it suffices to discuss minimization; every maximization problem can be converted via (3).

3 Types of Minima

Definition 3.1 (Local and Global Minima)**.** A minimum of f at x is called:

- **Global** if f assumes no smaller value on its entire domain.
- **Local** if there exists an open set U containing x such that $f(x)$ is a global minimum of f restricted to U.

4 Analytic Criteria for Local Minima

Proposition 4.1 (First- and Second-Order Conditions)**.** *Let $f\colon \mathbb{R}^d \to \mathbb{R}$ be twice continuously differentiable. Then x is a local minimum of f if*

$$\nabla f(x) = 0 \qquad \textit{and} \qquad H_f(x) \textit{ is positive definite}, \tag{4}$$

*where $H_f(x) = \left(\frac{\partial^2 f}{\partial x_i \, \partial x_j}(x)\right)_{i,j=1,\dots,d}$ is the **Hessian matrix** of f at x.*

Remark 4.2 (High-Dimensional Setting). In typical machine learning problems the dimension d is very large (millions in neural network training). We cannot plot or "look at" the function; we can only evaluate $f(x)$ point by point and use derivative information to guide the search.

5 Numerical Optimization

5.1 Basic Idea

Suppose we can compute the derivative $f'(x)$ at a point x. Then:

- The sign of $f'(x)$ tells us in which direction f decreases.
- The condition $f'(x) = 0$ identifies candidate minima (cf. Proposition 4.1).

Remark 5.1 (Minimization Strategy). Start at some point x_0. Choose a step size $c > 0$ and set

$$x_1 = x_0 - \operatorname{sign}\big(f'(x_0)\big) \cdot c.$$

A natural choice is $c = |f'(x_0)|$, yielding the update

$$x_1 = x_0 - f'(x_0). \tag{5}$$

Repeating this step leads to **gradient descent**.

5.2 Gradient Descent in One Dimension

Algorithm 5.2 (Gradient Descent, $d = 1$)**.** Given a differentiable function $f\colon \mathbb{R} \to \mathbb{R}$, start with a point $x_0 \in \mathbb{R}$ and fix a precision $\varepsilon > 0$.
Repeat for $n = 1, 2, \dots$:

1. If $|f'(x_n)| < \varepsilon$, report $x^* := x_n$ and terminate.
2. Otherwise, set
$$x_{n+1} := x_n - f'(x_n). \tag{6}$$

6 Derivatives in Multiple Dimensions

6.1 Reducing to One Dimension

Consider a function $f\colon \mathbb{R}^d \to \mathbb{R}$. To define a derivative at a point $x \in \mathbb{R}^d$:

1. Fix a direction vector $v \in \mathbb{R}^d$ and draw a line through x in direction v.
2. Intersect the graph of f with the vertical plane containing this line. The intersection is a one-dimensional function f_H, and we can compute its ordinary derivative $f'_H(x)$.
3. The value $f'_H(x)$ depends on the choice of direction v.

6.2 The Gradient

Definition 6.1 (Gradient)**.** Let $f\colon \mathbb{R}^d \to \mathbb{R}$ be differentiable. For each unit direction v, let $f'_v(x)$ denote the directional derivative of f at x in direction v. The **gradient** of f at x is the vector

$$\nabla f(x) := \Big(\frac{\partial f}{\partial x_1}(x),\ \ldots,\ \frac{\partial f}{\partial x_d}(x)\Big)^\top, \tag{7}$$

which points in the direction of steepest ascent and whose magnitude equals the maximal directional derivative:

$$\nabla f(x) = \arg\max_{\|v\|=1} f'_v(x).$$

Proposition 6.2 (Gradient–Contour Orthogonality)**.** *Let $C[f,c] := \{x \in \mathbb{R}^d \mid f(x) = c\}$ be a contour set of f. If $x \in C[f,c]$, then*

$$\nabla f(x) \perp C[f,c]. \tag{8}$$

Intuitively, the gradient points in the direction of maximal change, whereas along a contour line f does not change; locally, these two directions are orthogonal.

7 Gradient Descent in $\mathbb{R}^d$

Algorithm 7.1 (Basic Gradient Descent)**.** Given a differentiable function $f\colon \mathbb{R}^d \to \mathbb{R}$, start with $x_0 \in \mathbb{R}^d$ and fix $\varepsilon > 0$.
Repeat for $n = 1, 2, \ldots$:

1. If $\|\nabla f(x_n)\| < \varepsilon$, report $x^* := x_n$ and terminate.
2. Otherwise, set
$$x_{n+1} := x_n - \nabla f(x_n). \tag{9}$$

Algorithm 7.2 (Gradient Descent with Step Size)**.** A more general version introduces a step size (or learning rate) $\alpha(n) > 0$ that may depend on n:

$$x_{n+1} := x_n - \alpha(n)\,\nabla f(x_n). \tag{10}$$

The coefficient $\alpha(n)$ is called the **step size** in optimization, or the **learning rate** in machine learning.

Remark 7.3 (Gradient Descent and Local Minima). For smooth functions, gradient descent finds local minima. If the function is complicated, there may be no way to tell whether the solution is also a global minimum, since the derivative at both types of minima is zero.

Remark 7.4 (Saddle Points). In high dimensions, stationary points satisfying $\nabla f(x) = 0$ may be **saddle points** rather than local minima. At a saddle point, the Hessian $H_f(x)$ has both positive and negative eigenvalues (cf. Proposition 4.1). Gradient descent can slow down near saddle points, and distinguishing them from minima requires second-order information.

8 Newton's Method

Remark 8.1 (First- vs. Second-Order Methods). Gradient descent is a **first-order method**: it uses only the first derivative. Newton's method (or the Newton–Raphson algorithm) is a **second-order method** that also uses the second derivative. Higher-order methods converge in fewer steps, at the price of more computation per step.

8.1 Newton's Method for Root-Finding

Algorithm 8.2 (Newton's Method for Roots)**.** Newton's method finds a root of f, i.e. it solves $f(x) = 0$.
Start with $x_0 \in \mathbb{R}$ and fix $\varepsilon > 0$. Repeat for $n = 1, 2, \ldots$:

$$x_{n+1} := x_n - \frac{f(x_n)}{f'(x_n)}. \tag{11}$$

Terminate when $|f(x_n)| < \varepsilon$.

Example 8.3 (Square Root via Newton's Method)**.** To evaluate $g(a) = \sqrt{a}$, solve $f(x, a) = x^2 - a = 0$. Applying Algorithm 8.2 with this f is essentially how `sqrt()` is implemented in the standard C library.

8.2 Newton's Method for Minimization

Algorithm 8.4 (Newton's Method for Minima, $d = 1$)**.** We use Newton's root-finding method (Algorithm 8.2) to solve $f'(x) = 0$.
Start with $x_0 \in \mathbb{R}$ and fix $\varepsilon > 0$. Repeat for $n = 1, 2, \ldots$:

$$x_{n+1} := x_n - \frac{f'(x_n)}{f''(x_n)}. \tag{12}$$

Terminate when $|f'(x_n)| < \varepsilon$.

8.3 Newton's Method in Multiple Dimensions

Algorithm 8.5 (Newton's Method for Minima, $d > 1$)**.** The multi-dimensional generalisation replaces the scalar ratio by the inverse Hessian. Given $f\colon \mathbb{R}^d \to \mathbb{R}$ twice differentiable with invertible Hessian:

$$x_{n+1} := x_n - H_f^{-1}(x_n)\, \nabla f(x_n). \tag{13}$$

Compare with gradient descent (Algorithm 7.2):

$$x_{n+1} := x_n - \nabla f(x_n). \tag{14}$$

The Hessian correction adjusts the direction of the gradient step by taking into account the curvature of f at x_n.

Remark 8.6 (Newton Convergence Properties). Key properties of Newton's method:

- **Convergence guarantee:** The algorithm always converges if $f'' > 0$ (or H_f positive definite).
- **Two-phase convergence:** Far from the minimum the rate is linear; close to the minimum (where f is well-approximated by a quadratic), the rate becomes **quadratic** — roughly, the number of correct digits doubles at each step.
- **Dimension independence:** The number of Newton steps depends only weakly on d. Even in $\mathbb{R}^{10000}$, convergence to high precision typically requires only a handful of steps.
- **Caveat I:** Each step requires inverting the $d \times d$ Hessian, which can be very expensive.
- **Caveat II:** High-dimensional functions tend to have many more local minima; fast convergence does not address the question of *what* the algorithm converges *to*.

9 Constrained Optimization

Definition 9.1 (Constrained Optimization Problem)**.** An **optimization problem** for a function $f\colon \mathbb{R}^d \to \mathbb{R}$ is

$$\min_x \; f(x).$$

A **constrained optimization problem** adds requirements on x:

$$\min_x \; f(x) \qquad \text{subject to} \quad x \in G, \tag{15}$$

where $G \subset \mathbb{R}^d$ is the **feasible set**. The feasible set is often defined by constraints:

- An **equality constraint** $g(x) = 0$.
- An **inequality constraint** $g(x) \leq 0$.

9.1 Convexity

Definition 9.2 (Convex Set)**.** A set $A \subset \mathbb{R}^d$ is **convex** if, for every two points $x, y \in A$, the line segment connecting them lies entirely in A:

$$\lambda x + (1-\lambda)y \in A \qquad \text{for all } x, y \in A \text{ and } \lambda \in [0,1]. \tag{16}$$

Definition 9.3 (Convex Function)**.** A function f is **convex** if every line segment between function values lies above the graph of f:

$$\lambda\, f(x) + (1-\lambda)\, f(y) \geq f\big(\lambda x + (1-\lambda)y\big) \qquad \text{for all } x, y \text{ in } \operatorname{dom}(f) \text{ and } \lambda \in [0,1]. \tag{17}$$

Equivalently, f is convex if the epigraph $\{(x,t) \mid t \geq f(x)\}$ is a convex set. A twice differentiable function is convex if and only if

$$H_f(x) \succeq 0 \qquad \text{for all } x, \tag{18}$$

i.e. the Hessian is positive semidefinite everywhere.

Proposition 9.4 (Convexity and Optimization)**.** *If f is convex, then:*

1. *$\nabla f(x) = 0$ is a **sufficient** criterion for a minimum (cf. Proposition 4.1).*
2. *Every local minimum is global.*
3. *If f is **strictly convex** ($H_f(x) \succ 0$ for all x), the minimum is unique.*

9.2 Equality Constraints and Lagrange Multipliers

Definition 9.5 (Feasible Set for Equality Constraints)**.** Given a smooth function $g\colon \mathbb{R}^d \to \mathbb{R}$, the feasible set for the equality constraint $g(x) = 0$ is

$$G := \{x \in \mathbb{R}^d \mid g(x) = 0\}. \tag{19}$$

If g is sufficiently smooth, G is a smooth surface (manifold) in $\mathbb{R}^d$.

The key geometric insight is as follows. Let f_g denote the restriction of f to the surface G. At a constrained minimum of f, the gradient ∇f can be decomposed into a component $(\nabla f)_g$ tangent to G and a component $(\nabla f)_\perp$ normal to G. If f_g is minimised at a point on G, the tangential component must vanish: $(\nabla f)_g = 0$.

Proposition 9.6 (Lagrange Condition)**.** *At a constrained minimum, ∇f is orthogonal to G. By Proposition 6.2, the gradient ∇g is also orthogonal to G (since G is a contour set of g). Hence the two gradients are collinear:*

$$\nabla f(x) + \lambda \nabla g(x) = 0 \qquad \text{for some scalar } \lambda \neq 0. \tag{20}$$

Theorem 9.7 (Lagrange Multiplier Method)**.** *The constrained optimization problem*

$$\min_x \ f(x) \qquad \text{subject to} \quad g(x) = 0$$

is solved by finding (x, λ) satisfying the system

$$\begin{cases} \nabla f(x) + \lambda \nabla g(x) = 0, \\ g(x) = 0. \end{cases} \tag{21}$$

Since ∇f and ∇g are d-dimensional, this is a system of $d+1$ equations in $d+1$ unknowns $(x_1, \ldots, x_d, \lambda)$. The scalar λ is called the ***Lagrange multiplier****.*

9.3 Inequality Constraints

Definition 9.8 (Inequality-Constrained Problem)**.** For $f\colon \mathbb{R}^d \to \mathbb{R}$ and a convex function g, the problem

$$\min_x \ f(x) \qquad \text{subject to} \quad g(x) \leq 0 \tag{22}$$

is an optimization problem with **inequality constraint**. The feasible set is $G := \{x \mid g(x) \leq 0\}$.

Remark 9.9 (Interior vs. Boundary). The feasible set decomposes as $G = \operatorname{int}(G) \cup \partial G$, where the interior is given by $g(x) < 0$ and the boundary by $g(x) = 0$. Two cases arise:

1. **Minimum in the interior:** Here $f_g = f$, so the unconstrained criterion $\nabla f(x) = 0$ applies (cf. Proposition 4.1).
2. **Minimum on the boundary:** Since $g(x) = 0$, the geometry is the same as for equality constraints (Theorem 9.7), but the sign of λ matters.

Proposition 9.10 (Sign Condition on the Boundary)**.** *An extremum on the boundary of G is a minimum only if ∇f points into G. Since ∇g points away from G (in the direction of increasing g), the two gradients must be anti-parallel:*

$$\nabla f(x) = -\lambda \nabla g(x) \qquad \text{with} \quad \lambda > 0. \tag{23}$$

Remark 9.11 (Combining the Cases). The combined conditions are:

$$\nabla f = -\lambda \nabla g, \qquad g(x) \leq 0, \qquad \lambda = 0 \text{ if } x \in \operatorname{int}(G), \qquad \lambda > 0 \text{ if } x \in \partial G.$$

Noting that $g(x) < 0$ in the interior and $g(x) = 0$ on the boundary, in either case we have $\lambda \cdot g(x) = 0$. The case distinction can therefore be replaced by the compact conditions $\lambda\, g(x) = 0$ and $\lambda \geq 0$.

9.4 KKT Conditions

Theorem 9.12 (Karush–Kuhn–Tucker Conditions)**.** *The inequality-constrained problem (Definition 9.8)*

$$\min_x \ f(x) \qquad \text{subject to} \quad g(x) \leq 0$$

can be solved by finding (x, λ) *satisfying the* ***Karush–Kuhn–Tucker (KKT) conditions****:*

$$\boxed{\begin{aligned} \nabla f(x) + \lambda \nabla g(x) &= 0, && \textit{(stationarity)} \\ \lambda\, g(x) &= 0, && \textit{(complementary slackness)} \\ g(x) &\le 0, && \textit{(primal feasibility)} \\ \lambda &\ge 0. && \textit{(dual feasibility)} \end{aligned}} \tag{24}$$

Remark 9.13 (Interpretation of the KKT Conditions). Each condition serves a specific purpose:

Condition	**Ensures that...**	**Purpose**
$\nabla f + \lambda \nabla g = 0$	If $\lambda = 0$: $\nabla f = 0$	Optimality inside G
	If $\lambda > 0$: $\nabla f \parallel -\nabla g$	Optimality on ∂G
$\lambda\, g(x) = 0$	$\lambda = 0$ in interior	Distinguish int(G) and ∂G
$\lambda \ge 0$	∇f cannot align with ∇g	Boundary optimum is minimum

Remark 9.14 (Why g Should Be Convex). If g is convex, then the feasible set $G = \{x \mid g(x) \le 0\}$ is convex (cf. Definition 9.2). If G is not convex, the KKT conditions (Theorem 9.12) remain *necessary* but are no longer *sufficient*: multiple points may satisfy (24), only one of which is the true minimum.

10 Interior Point Methods

Remark 10.1 (Numerical Solution of Constrained Problems). After transforming a constrained problem using Lagrange multipliers or KKT conditions (Theorem 9.7, Theorem 9.12), we still need to solve the resulting system numerically.

Definition 10.2 (Barrier Function)**.** Consider the problem $\min_x f(x)$ subject to $g(x) < 0$. The constraint can be encoded via an indicator function:

$$\min_x \; f(x) + C \cdot \mathbf{1}_{[0,\infty)}\big(g(x)\big),$$

but the indicator is not differentiable at 0. A **barrier function** approximates the indicator by a smooth function. A standard choice is the **log barrier**:

$$\beta_t(x) := -\frac{1}{t} \log(-x), \qquad t > 0, \tag{25}$$

which satisfies $\beta_t(x) \to \infty$ as $x \nearrow 0$.

Algorithm 10.3 (Interior Point Method)**.** To solve

$$\min_x \; f(x) \qquad \text{subject to} \quad g_i(x) < 0, \quad i = 1, \dots, m,$$

we instead solve the unconstrained problem

$$\min_x \; f(x) + \sum_{i=1}^{m} \beta_{i,t}\big(g_i(x)\big), \tag{26}$$

with one barrier function $\beta_{i,t}$ (Definition 10.2) per constraint. The parameter t is increased progressively, tightening the approximation. The inner minimisation is typically performed by Newton's method (Algorithm 8.5).

Remark 10.4 (General Strategy for Constrained Problems). A common pipeline for constrained optimization:

1. Convert constraints into a tractable form using Lagrange multipliers (Theorem 9.7) or KKT conditions (Theorem 9.12).
2. Replace remaining constraints by barrier functions (Definition 10.2).
3. Apply numerical optimization (typically Newton's method, Algorithm 8.5).

11 Relevance of Convexity

Remark 11.1 (Convexity as a Generalisation of Linearity). A common claim is that convexity matters because convex functions have a unique global minimum (Proposition 9.4). While correct, there is a deeper reason. A numerical optimisation algorithm can only query a function at finitely many points. Convexity allows us to draw **global conclusions from local information**:

- **Constant functions:** Knowing $f(x)$ at one point determines f everywhere.
- **Linear functions:** Knowing $f(x)$ and $\nabla f(x)$ at one point determines f everywhere.
- **Convex functions:** Knowing $f(x)$ and $\nabla f(x)$ at one point tells us in which half-space the minimum lies.

Various other global properties of convex functions are determined by their local behaviour; some are much deeper and more surprising than the above.

12 Stochastic Gradient Descent

Definition 12.1 (Stochastic Gradient)**.** If ε is a random variable with $\mathbb{E}[\varepsilon] = 0$, then

$$\widehat{\nabla} f(x) := \nabla f(x) + \varepsilon \tag{27}$$

is called a **stochastic gradient** of f at x.

Algorithm 12.2 (Stochastic Gradient Descent (SGD))**.** Substituting the stochastic gradient (Definition 12.1) into gradient descent with step size $\alpha\colon \mathbb{N} \to \mathbb{R}_+$ (cf. Algorithm 7.2):

$$\hat{x}_{n+1} = \hat{x}_n - \alpha(n)\, \widehat{\nabla} f(\hat{x}_n). \tag{28}$$

12.1 Convergence of SGD

Remark 12.3 (Comparison with Gradient Descent). Fix x_0. A gradient descent sequence and an SGD sequence satisfy

$$x_{n+1} = x_0 - \sum_{i=1}^{n} \alpha(i)\, \nabla f(x_i), \qquad \hat{x}_{n+1} = x_0 - \sum_{i=1}^{n} \alpha(i)\, \widehat{\nabla} f(\hat{x}_i). \tag{29}$$

Writing $\widehat{\nabla} f(\hat{x}_i) = \nabla f(x_i) + \delta_i$, we obtain

$$\hat{x}_{n+1} = x_{n+1} - \sum_{i=1}^{n} \alpha(i)\, \delta_i. \tag{30}$$

Note that $\delta_i \neq \varepsilon_i$ in general. Even if the noise terms $\varepsilon_1, \varepsilon_2, \dots$ are i.i.d., the deviations $\delta_1, \delta_2, \dots$ are typically not, since the SGD iterates $\hat{x}_i$ differ from the GD iterates x_i.

Theorem 12.4 (Robbins–Monro Conditions)**.** *Sufficient conditions on the step sizes for SGD convergence (under appropriate regularity assumptions) are the* ***Robbins–Monro conditions****:*

$$\sum_{n=1}^{\infty} \alpha(n) = \infty \qquad \textit{and} \qquad \sum_{n=1}^{\infty} \alpha(n)^2 < \infty. \tag{31}$$

- *The first condition ensures that points arbitrarily far from x_0 are reachable. This is typically also required for ordinary gradient descent in $\mathbb{R}^d$.*

- *The second condition ensures finite total variance.*

These conditions are only meaningful together with assumptions on the dependence structure of the noise $\varepsilon_1, \varepsilon_2, \ldots$

Remark 12.5 (The Variance Condition). To illustrate the role of (31), suppose $\delta_1, \delta_2, \ldots$ are i.i.d. with variance σ^2. Then

$$\operatorname{Var}\big[\alpha(n)\delta_n\big] = \alpha(n)^2\,\sigma^2 \qquad \text{and} \qquad \operatorname{Var}\Big[\sum_{n=1}^{\infty} \alpha(n)\,\delta_n\Big] = \Big(\sum_{n} \alpha(n)^2\Big)\sigma^2.$$

The second Robbins–Monro condition (Theorem 12.4) guarantees that this total variance is finite.

12.2 SGD in Machine Learning

Remark 12.6 (Additive Objectives and Mini-Batches). For an additive objective over n data points, the gradient cost scales with sample size:

$$f(X_1, \ldots, X_n, \theta) = \frac{1}{n}\sum_{i=1}^{n} f(X_i, \theta) \qquad \Longrightarrow \qquad \nabla_\theta f = \frac{1}{n}\sum_{i=1}^{n} \nabla_\theta f(X_i, \theta). \tag{32}$$

A **mini-batch** is a random subset $\tilde{X}_1, \ldots, \tilde{X}_k$ of the full dataset ($k < n$). Computing the gradient only on the mini-batch yields a stochastic gradient (Definition 12.1):

$$\widehat{\nabla}_\theta f(\tilde{X}_1, \ldots, \tilde{X}_k, \theta) = \nabla_\theta f(X_1, \ldots, X_n, \theta) + \varepsilon, \tag{33}$$

where ε is a zero-mean random variable arising from the random selection of the mini-batch.

References

- S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
- H. Robbins and S. Monro, "A stochastic approximation method", *Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein, "Visualizing the loss landscape of neural nets", arXiv:1712.09913, 2017.